END-TO-END REQUEST TRACE

Continuum OS: Prompt to Token, Traced

A single client request followed hop by hop through the built cluster — api-gatewayd, schedulerd, shard-execd, and back — with the exact wire messages, ports, and struct fields involved at every step.

Doc	ARCH-005
Status	Grounded in the built codebase at ~/continuum-os
Date	2026-09-19
Companion to	ARCH-001 (architecture), ARCH-002 (hardware/build), ARCH-003 (runtime diagram), ARCH-004 (deployment & model loading)

Copyright 2026 Christopher Hinds, Stratum Technologies LLC. Licensed under the Apache License, Version 2.0 (SPDX: Apache-2.0).

Contents

00 Scope — the request being traced

01 Client to api-gatewayd — HTTP in

02 api-gatewayd to schedulerd — WIRE_SCHED_REQUEST

03 schedulerd to shard-execd — WIRE_DISPATCH_PROMPT

04 shard-execd — prefill

05 shard-execd — decode loop, one token at a time

06 The return path — WIRE_TOKEN_CHUNK back to the client

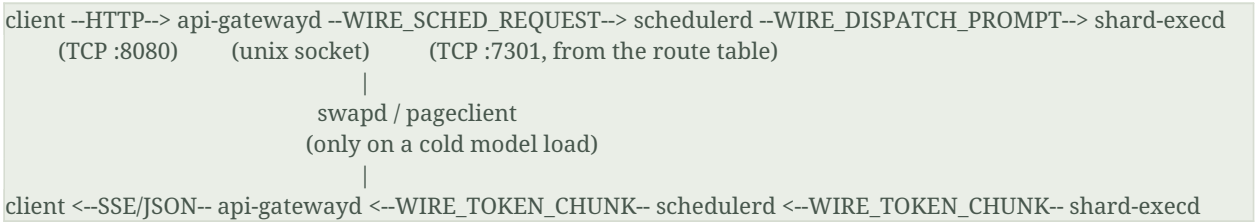
07 What doesn't happen on a warm request

08 Concurrency and batching — what this build actually does

Copyright 2026 Christopher Hinds, Stratum Technologies LLC. Licensed under the Apache License, Version 2.0 (SPDX: Apache-2.0).

00 Scope — the request being traced

This document traces one concrete request — a client's POST /v1/chat/completions — through every process hop in the built cluster, from the moment it lands on api-gatewayd's listen socket to the moment the last token byte reaches the client. Every message type, port, and struct field named below is copied from the real source tree at ~/continuum-os, not the abstract architecture — services/api-gatewayd/src/main.c, services/schedulerd/src/main.c, services/shard-execd/src/main.c, and the shared wire definitions in lib/wire/include/wire.h and protocols/.



> **SIX HOPS, THREE PROCESSES, ONE CONNECTION AT A TIME**
>
> Every daemon in this path — gateway, scheduler, shard executor — handles one connection fully before accepting the next (see each service's own module comment). There is no request multiplexing or batching anywhere in this reference build, so the trace below is exactly what happens for one request, with nothing else interleaved on the same shard-execd.

01 Client to api-gatewayd — HTTP in

POST /v1/chat/completions arrives on TCP :8080 — api-gatewayd.conf's listen_port. handle_connection() parses the request with the vendored picohttpparser, buffering via conn_buf_read_more() until the full Content-Length body has arrived, then hands off to handle_chat_completions().

Parsing the request body

parse_chat_request() walks the JSON with the vendored jsmn tokenizer:

- model — the target model name, copied verbatim (max 31 chars)
- messages[].content — every message's content string is concatenated into one \n-joined prompt buffer. There is no chat template and no real subword tokenizer: api-gatewayd's own module comment calls "tokenization" the identity function on UTF-8 bytes

Copyright 2026 Christopher Hinds, Stratum Technologies LLC. Licensed under the Apache License, Version 2.0 (SPDX: Apache-2.0).

- max_tokens (default 64), temperature (default 0.8), stream (default false)

> **IF `STREAM:TRUE` WAS REQUESTED**
>
> The gateway writes SSE response headers (Content-Type: text/event-stream) back to the client immediately — before it has generated a single token. From this point the connection is committed to streaming; every downstream error becomes an SSE data: {"error":...} event followed by data: [DONE], not an HTTP error status.

02 api-gatewayd to schedulerd — WIRE_SCHED_REQUEST

dispatch_to_scheduler() opens a Unix socket to cfg.scheduler_unix_socket (/run/continuum/schedulerd.sock) and sends one WIRE_SCHED_REQUEST (wire type 10) frame.

What's on the wire

A sched_request_header_t immediately followed by the prompt — one uint32_t per raw UTF-8 byte, not a real token id:

```
struct sched_request_header_t {          // protocols/sched_protocol.h
  char     model_name[32];
  uint32_t n_prompt_tokens;
  uint32_t max_new_tokens;
  float    temperature;
  uint32_t rng_seed;                     // derived from a stream_id: &req XOR time()
};                                 // n_prompt_tokens x uint32_t token ids follow
```

The stream_id used for this rng_seed is also what threads the whole request through every later frame — schedulerd and shard-execd both echo it back unchanged, which is how the gateway would (in a concurrent build) match a reply to its request; in this synchronous, one-connection-at-a-time reference build it's carried faithfully but never actually needed for demuxing.

03 schedulerd to shard-execd — WIRE_DISPATCH_PROMPT

schedulerd reads the frame off its Unix listener and calls scheduler_find_route() — a flat table straight from schedulerd.conf, e.g. route toy-model 10.0.0.21:7301. No load-based or automatic placement: it's a static lookup by model name.

> **NO ROUTE FOUND**
>
> schedulerd sends WIRE_ERROR straight back with the payload "model not found". The

Copyright 2026 Christopher Hinds, Stratum Technologies LLC. Licensed under the Apache License, Version 2.0 (SPDX: Apache-2.0).

> gateway turns that into an SSE error+[DONE] (streaming) or a 502 (non-streaming) — see handle_chat_completions()'s WIRE_ERROR branch.

On a hit, schedulerd opens a new TCP connection to that shard-execd's port. This is the one hop in the whole trace that crosses physical boards — control-plane and compute nodes are different Pi 5s per ARCH-002 §01 — and it's worth noting explicitly because it wasn't always TCP: bug #5 in the project's own bug list was schedulerd originally dialing shard-execd over a Unix socket, which silently can't work once the two processes are on different machines. The fix is what's traced here.

dispatch_prompt_header_t is deliberately field-compatible with sched_request_header_t past the model name, so forwarding the token array is a byte-range memcpy, not a re-parse:

```
struct dispatch_prompt_header_t {        // protocols/dispatch_protocol.h
  uint32_t n_prompt_tokens;
  uint32_t max_new_tokens;
  float    temperature;                  // <=0 means greedy argmax
  uint32_t rng_seed;
};                                 // n_prompt_tokens x uint32_t token ids follow
```

Sent as WIRE_DISPATCH_PROMPT (wire type 9).

04 shard-execd — prefill

handle_connection() in services/shard-execd/src/main.c reads the dispatch frame. Two things happen before any matrix multiply runs.

01 Read the WIRE_DISPATCH_PROMPT frame; reject anything else with a warning log

02 kvcache_lookup_prefix() checks this prompt's leading tokens against a previously cached sequence — logged if matched, but NOT used to skip recomputation (observability only in this reference build)

03 Allocate a fresh kv_session_t sized for n_prompt_tokens + max_new_tokens

04 For every PROMPT token, in order: toy_model_forward_step() — full causal self-attention + FFN, appending this step's key/value into the session history

05 Only the logits from the LAST prompt token are kept — prefill exists purely to build up KV history, not to produce output

Fig. 1 — everything shard-execd does before generating a single token.

> **MODEL TENSORS ARE ALREADY RESIDENT**
> The model's eight tensors were faulted into this node's shared DRAM pool once, at shard-execd's own startup, by toy_model_load() -> pageclient_get_page() -> swapd's userfaultfd path (traced fully in ARCH-004 §05). A normal request does not re-page anything — every

Copyright 2026 Christopher Hinds, Stratum Technologies LLC. Licensed under the Apache License, Version 2.0 (SPDX: Apache-2.0).

> forward step here is a float matmul against tensors already sitting in this process's memory. See §07 for the one case where paging re-enters the picture.

05 shard-execd — decode loop, one token at a time

For each of up to max_new_tokens steps, toy_model_forward_step() + toy_model_sample() + one WIRE_TOKEN_CHUNK send, in a tight loop:

01 toy_model_sample(logits, vocab_size, temperature, &rng_state) picks the next token — argmax if temperature<=0, otherwise an inverse-CDF sample against a xorshift32 PRNG seeded from the request's rng_seed

02 Append the token to full_seq (kept for the prefix-cache insert at the end)

03 Decide is_last: either this is the final requested token, or the sampled token equals 0 (token 0 doubles as EOS in this byte-level vocabulary)

04 Send WIRE_TOKEN_CHUNK (wire type 5, payload = one uint32_t token) IMMEDIATELY over the same TCP connection back to schedulerd — WIRE_FLAG_FINAL set iff is_last

05 If not last: run toy_model_forward_step() once more with this token to produce the NEXT step's logits, and loop

Fig. 2 — the decode loop, from toy_model_sample()'s call site in shard-execd's handle_connection().

After the loop exits, kvcache_insert() stores the full generated sequence (prompt + completion) into the prefix cache under opaque page-handle stand-ins, for a future request's prefix match to find.

06 The return path — WIRE_TOKEN_CHUNK back to the client

Each token chunk crosses two more relays before it reaches the client — neither buffers.

schedulerd: pure relay, no batching

The same handle_connection() call that dispatched the prompt now loops reading WIRE_TOKEN_CHUNK frames off the shard-execd TCP socket and re-sends each one — same type and flags, re-stamped with the original stream_id — to the gateway over the Unix socket, stopping the instant it sees WIRE_FLAG_FINAL. Nothing is accumulated; each frame is forwarded before the next one is even read.

api-gatewayd: SSE or buffered JSON

The gateway's loop in handle_chat_completions() reads each chunk the same way and branches on whether the original request asked for stream:

Copyright 2026 Christopher Hinds, Stratum Technologies LLC. Licensed under the Apache License, Version 2.0 (SPDX: Apache-2.0).

Mode	What happens per token	What the client sees
stream:true	JSON-escape the single byte, write one SSE event immediately: data: {"id":...,"choices":[{"delta":{"content":"<byte>"},"finish_reason":null}]}\n\n	Bytes arrive one at a time, roughly one forward-pass apart; final chunk carries "finish_reason":"stop", followed by data: [DONE]\n\n
stream:false	Byte appended to a local buffer (collected), nothing written yet	Nothing until the final chunk, then one chat.completion JSON body with the full message.content string

TIME-TO-FIRST-BYTE, CONCRETELY

In the streaming case, the client's first visible byte is bounded by: one full prefill pass over the prompt (§04) + one decode step (§05) + three hops of frame relay (shard-execd -> schedulerd -> gateway -> client), all synchronous. Nothing in this path overlaps token generation for request N with anything else — see §08.

07 What doesn't happen on a warm request

Worth stating explicitly, since it's easy to assume more machinery is in the hot path than actually is:

- No re-paging. Tensors are resident from process startup; the decode loop touches only already-mapped memory, unless swapd's LRU sweep (paging_engine_evict_lru) evicted a page for memory pressure between requests — then the next pageclient_get_page() call transparently re-faults it, adding one swapd round-trip before that tensor's next use
- No prefix-cache reuse. The lookup in §04 runs and logs a match count, but recomputation is never skipped — every prompt token is fully re-run through prefill regardless of cache hits
- No cross-node hop inside shard-execd. Both prefill and decode run entirely on the one compute node the scheduler routed to; there is no pipeline-parallel activation hand-off (WIRE_ACTIVATION_HOP exists in wire.h but nothing in this path emits it) — the whole model lives on one shard in this reference build
- No batching of concurrent requests. See §08

Copyright 2026 Christopher Hinds, Stratum Technologies LLC. Licensed under the Apache License, Version 2.0 (SPDX: Apache-2.0).

08 Concurrency and batching — what this build actually does

Every one of the three daemons in this trace uses the same shape: accept(), handle that connection fully and synchronously, close(), accept() again. shard-execd's own module comment is explicit about why: ARCH-002's scheduler-side batching is what would normally keep a shard busy across concurrent requests, and this reference build exists to prove the paging-plus-forward-pass path end to end, not to build a production request scheduler inside shard-execd.

> **PRACTICAL CONSEQUENCE**
>
> A second client request arriving while this one is mid-decode simply blocks at accept() on whichever daemon it reaches first — there is no queueing visible to that client beyond the OS socket backlog (sock_tcp_listen/sock_unix_listen's backlog argument, 8-32 depending on the service). This is a known, named simplification, not a bug: it isolates the one mechanism this reference build set out to prove.

Copyright 2026 Christopher Hinds, Stratum Technologies LLC. Licensed under the Apache License, Version 2.0 (SPDX: Apache-2.0).